Republic of Tunisia
Ministry of Higher Education
and Scientific Research

University of Sfax

National School of Engineering
of Sfax



Department :
Computer Engineering
and Applied Mathematics

End of year project Report

Presented at

National School of Engineering of Sfax

Section

Computer Engineering and Applied Mathematics

Presented by :

Hamza Hentati
Nejmeddin Ben Maatoug
Bachar Tounsi

Presented at :

26 / 04 / 2025

Phishing Attacks Detection using Machine Learning

Academic Supervisor : Mrs. Maha Charfeddine

Jury : Mrs. Mouna Baklouti

Academic Year : 2024-2025

Appreciation

We would like to express our sincere gratitude to our academic supervisor, **Mrs. Maha Charfeddine**, for her invaluable guidance, continuous support, and encouragement throughout the development of this project. Her expertise, insightful advice, and constructive feedback have been instrumental in shaping this work.

Her dedication and availability have greatly contributed to our learning experience, and We truly appreciate her patience and willingness to share her knowledge. This project would not have been possible without her supervision and mentorship.

We would also like to extend our heartfelt thanks to the members of the jury for their time, interest, and valuable feedback. Their presence and thoughtful evaluation added great value to our work.

Finally, we are grateful to everyone who attended our presentation. Your presence and support meant a lot to us and made the moment even more meaningful.

Thank you all for your encouragement and for contributing to the success of this project.

TABLE OF CONTENTS

LIST OF FIGURES v

GENERAL INTRODUCTION 1

1 Project Context and Specifications 2

1.1 INTRODUCTION . . . 3
1.2 Project Overview . . . 3
1.3 Problem Statement . . . 4
1.3.1 Understanding Phishing Attacks . . . 4
1.3.2 Phishing Detection . . . 4
1.3.3 Existing Phishing Detection Techniques . . . 4
1.4 Machine Learning . . . 6
1.5 Deep Learning . . . 6
1.6 Ensemble Learning . . . 6
1.7 Proposed Solution . . . 7
1.8 Project Requirements . . . 8
1.8.1 Definition of Actors . . . 8
1.8.2 Functional Requirements . . . 8
1.8.3 Non-functional requirements . . . 10
1.9 Use Case Diagram . . . 11
1.10 Activity Diagram . . . 12
1.11 Methodology . . . 12
1.11.1 Crisp-DM Methodology . . . 13
1.11.2 Crisp-DM Stages . . . 13
1.12 Conclusion . . . 14

2 Methodology 15

2.1 Introduction . . . 16
2.2 Project Architecture . . . 16

2.3 Dataset Understanding 17
2.3.1 Dataset Exploration 17
2.3.1.1 The Web Page Phishing Detection dataset 17
2.3.1.2 Phishing Dataset for Machine Learning: Feature Evaluation . 17
2.3.1.3 Phishing Websites UC irvine 18
2.3.1.4 PhishFOE Dataset 18
2.3.2 Dataset Selection 18
2.4 Data Preprocessing 20
2.4.1 Data Cleaning 20
2.4.2 Feature Scaling 20
2.4.3 Anomaly Detection 21
2.4.4 Data Balancing 21
2.4.5 Feature Selection 23
2.5 Model Selection and Traning 23
2.5.1 Convolutional Neural Networks Model (CNNs) 23
2.5.2 Random Forest Model 24
2.5.3 Light Gradient Boosting Machine Model (LightGBM) 24
2.5.4 Combination of two Models (LightGBM , CNN) 25
2.5.5 Other Models 25
2.5.5.1 Transformers 25
2.5.5.2 Multilayer Perceptron (MLP) 25
2.5.5.3 Long Short-Term Memory (LSTM) 26
2.5.5.4 Extreme Gradient Boosting (XGBoost) 26
2.5.5.5 Hybrid Model: XGBoost + LSTM 26
2.6 Evaluation Metrics 27
2.6.1 Accuracy Metric 27
2.6.2 Precision Metric 27
2.6.3 Recall Metric (Sensitivity) 28
2.6.4 F1-Score Metric 28
2.7 Conclusion 28

3 Experimental Results and Deployment 29
3.1 INTRODUCTION 30
3.2 Software Environment 30
3.2.1 Development environment : Google Colab 30
3.2.2 Development environment : Visual Studio Code 30

3.2.3 Python Library : Streamlit 30
3.3 Results and Evaluation 31
3.3.1 Balancing Techniques 31
3.3.2 Models Details 32
3.4 Feature Selection Methods 33
3.4.1 Combining Models with RFE-CV 33
3.4.2 Combining Models with f-classif 34
3.4.3 Comparing f-classif and RFE-CV 34
3.5 Other Results 35
3.5.1 Comparing our results with other studies 35
3.5.2 Additional Models Tested on The phishFOE dataset 36
3.6 Deployment 37
3.6.1 System Workflow Overview 37
3.6.2 Deployment Environment Preparation 38
3.6.3 User Interface and Prediction Output 39
3.6.4 Notable Technical Features 41
3.6.5 Conclusion 42

GENERAL CONCLUSION **43**

LIST OF FIGURES

1.1 UseCase Diagram 11
1.2 Activity Diagram 12
1.3 CRISP-DM stages [4] 14

2.1 Project Architecture Diagram 16
2.2 Structure of PhishFOE Dataset 19
2.3 Class Distribution 20
2.4 Class Distribution Before and After SMOTE 22

3.1 Activity Diagram 33
3.2 Performance comparison of various models using f-classif 34
3.3 Comparaison between f-classif and RFE-CV 35
3.4 Deployment Design 37
3.5 LightGBM Model Performance Dashboard 39
3.6 URL Legitimacy Analysis Result – Legitimate URL 40
3.7 URL Legitimacy Analysis Result – Phishing URL 40
3.8 URL Legitimacy Prediction Dashboard 41

GENERAL INTRODUCTION

With the rapid growth of digital technologies and increased reliance on electronic communication, cybersecurity has become a major concern. Among the most common and sophisticated threats is phishing a social engineering attack where cybercriminals impersonate trusted entities (banks, companies, or government agencies) to trick users into revealing sensitive information.

Phishing attacks mainly occur through emails designed to look legitimate, often containing malicious links or attachments. These messages exploit emotions like urgency or fear to manipulate users, potentially leading to fraud, identity theft, or large-scale data breaches.

Traditional defenses, such as spam filters and rule-based systems, have limitations, as attackers constantly evolve their tactics. In response, artificial intelligence and machine learning offer promising solutions by detecting phishing patterns and continuously improving accuracy.

This project aims to build an intelligent phishing email detection system using machine learning algorithms. We analyze datasets of phishing and legitimate emails, extract relevant features, and train classification models. Ensemble Learning techniques (Bagging, Boosting, Stacking) are also employed to enhance robustness.

This report outlines our methodology, technical choices, performance evaluation, and improvement opportunities contributing to the growing role of machine learning in strengthening cybersecurity.

Chapter

1 Project Context and Specifications

Contents

1.1 INTRODUCTION 3

1.2 Project Overview 3

1.3 Problem Statement 4

1.3.1 Understanding Phishing Attacks 4

1.3.2 Phishing Detection 4

1.3.3 Existing Phishing Detection Techniques 4

1.4 Machine Learning 6

1.5 Deep Learning 6

1.6 Ensemble Learning 6

1.7 Proposed Solution 7

1.8 Project Requirements 8

1.8.1 Definition of Actors 8

1.8.2 Functional Requirements 8

1.8.3 Non-functional requirements 10

1.9 Use Case Diagram 11

1.10 Activity Diagram 12

1.11 Methodology 12

1.11.1 Crisp-DM Methodology 13

1.11.2 Crisp-DM Stages 13

1.12 Conclusion 14

1.1 INTRODUCTION

In this chapter, we explore the context and specifications of our phishing detection project. We begin by analyzing different types of phishing attempts and identifying key features used to detect them, examining existing research in this domain that leverages various datasets and machine learning techniques. These studies provide valuable reference points that will be used to compare and assess our approach in the following chapters. Furthermore, we outline the functional and non-functional requirements, define the key actors involved, and establish the boundaries of our research. By combining a thorough understanding of the problem domain with detailed technical specifications, we ensure that our approach is both theoretically sound and practically implementable, addressing the growing threat of phishing attacks through effective machine learning solutions.

1.2 Project Overview

Our project focuses on developing an Artificial Intelligence model for accurately detecting phishing websites, thereby strengthening cybersecurity defenses in the digital space. The process begins with collecting a diverse dataset that includes both legitimate and fraudulent websites. Before training the model, we apply extensive preprocessing techniques to ensure data quality and consistency. Using advanced machine learning algorithms, the model learns to differentiate between legitimate and phishing websites. Once trained, the model is integrated into a user-friendly interface, enabling real-time classification of websites. Ultimately, this project represents a comprehensive effort to harness Artificial Intelligence for proactive phishing detection, enhancing online security for users worldwide.

1.3 Problem Statement

1.3.1 Understanding Phishing Attacks

Phishing is a form of cyberattack where malicious actors impersonate trusted individuals, organizations, or institutions to deceive victims into revealing sensitive information such as passwords, credit card details, or personal data. These attacks are typically executed through fraudulent emails, messages, or websites that closely mimic legitimate sources, tricking users into providing confidential information or clicking on harmful links. The primary objective of phishing is to exploit human trust and psychological manipulation to gain unauthorized access to sensitive data, which can be used for identity theft, financial fraud, or system breaches.

1.3.2 Phishing Detection

Detecting phishing attempts involves identifying and preventing cybercriminals from misleading individuals into disclosing private data, such as login credentials or financial details, through deceptive digital communications. This requires the use of technological tools, algorithms, and user vigilance to recognize phishing indicators, including suspicious URLs, mismatched sender addresses, and misleading content. A combination of automated detection mechanisms and user awareness plays a crucial role in minimizing the risk of falling victim to phishing scams.

1.3.3 Existing Phishing Detection Techniques

Several strategies have been developed to counter phishing attacks, each leveraging different methodologies to enhance security and minimize risks. These techniques range from traditional rule-based approaches to more advanced artificial intelligence-driven methods.

- **Blacklist-Based Detection [1]:** This traditional approach relies on maintaining a database of known malicious URLs. Security systems such as antivirus software, intrusion detection systems, and spam filters use these blacklists to block access to flagged websites, preventing

users from interacting with harmful content. However, this method is limited as it fails to detect newly created phishing sites that are not yet listed.

- **Heuristic-Based Detection:** Instead of relying on predefined blacklists, heuristic-based detection examines the structure and characteristics of emails, URLs, and web pages to identify potential threats. This approach analyzes domain names, webpage elements, and keyword patterns commonly found in phishing attacks. It offers a more flexible solution but may produce false positives if not fine-tuned properly.

- **Domain Name System (DNS) Filtering [2]:** This technique restricts access to malicious websites by filtering domains and IP addresses associated with phishing activities.
  Organizations use DNS filtering to block suspicious web traffic and prevent users from visiting fraudulent sites. It is particularly effective in enterprise environments where administrators can control browsing activities across networks.

- **Rule-Based Detection [3]:** This method applies predefined rules to identify phishing attempts based on specific characteristics found in email headers, website content, and URLs. By analyzing patterns such as misspelled domain names, misleading sender addresses, and urgent language used in phishing emails, rule-based detection systems can flag suspicious messages or web pages. However, cybercriminals can bypass these rules by frequently altering their phishing strategies.

- **User Awareness and Training:** Educating users about phishing tactics, warning signs, and security best practices is essential in mitigating phishing risks. Many organizations conduct regular cybersecurity training programs, including simulated phishing attacks, to test employees' ability to recognize fraudulent emails and report them.
  Awareness campaigns can significantly reduce successful phishing attempts by fostering a security-conscious culture.

In our project,by combining multiple phishing detection techniques, we focus on leveraging AI techniques to enhance phishing detection capabilities, aiming for a more proactive and automated approach to online security.

1.4 Machine Learning

Machine Learning [19] (ML), a core branch of artificial intelligence, explores how computational systems can enhance their understanding, decision-making, and performance through exposure to data and experience. Unlike traditional programming, where rules are explicitly defined, ML enables systems to learn and adapt autonomously. This interdisciplinary field integrates principles from computer science, statistical analysis, cognitive psychology, neural biology, economic modeling, and systems control to develop algorithms that improve with iterative learning. By leveraging these diverse disciplines, ML empowers machines to refine their predictions, recognize complex patterns, and optimize actions without direct human intervention.

1.5 Deep Learning

Deep Learning [34] (DL), a specialized subfield of machine learning, focuses on algorithms inspired by the structure and function of the human brain, known as artificial neural networks. These models are designed to automatically learn hierarchical representations from vast amounts of data, enabling machines to perform complex tasks such as image recognition, natural language understanding, and autonomous decision-making. Unlike traditional ML approaches that rely heavily on manual feature extraction, DL systems excel at discovering intricate patterns and abstractions directly from raw data.

1.6 Ensemble Learning

Ensemble learning [25] is a machine learning technique that combines multiple models to improve predictive performance. Instead of relying on a single model, ensemble methods integrate the strengths of various models to achieve better accuracy and robustness. The three primary types of ensemble learning are:

a) Bagging (Bootstrap Aggregating)

This method involves training multiple models independently on different random subsets of the data and then aggregating their predictions. Bagging helps reduce variance and prevent overfitting.

b) Boosting

Boosting trains models sequentially, where each new model focuses on correcting the errors made by the previous ones. This approach aims to reduce bias and improve overall model performance. Algorithms like AdaBoost and Gradient Boosting are popular examples of boosting techniques.

c) Stacking (Stacked Generalization)

Stacking combines multiple diverse models by training a meta-model to learn how to best combine the base models' predictions. This method leverages the strengths of different algorithms to improve predictive accuracy.

1.7 Proposed Solution

Our proposed solution leverages advanced machine learning and deep learning techniques to create a robust phishing detection system capable of identifying fraudulent websites with high accuracy. By combining the strengths of different algorithms and approaches, we aim to develop a comprehensive system that can:

1. Extract and analyze key features from URLs and webpage content
2. Apply sophisticated classification models to determine website legitimacy
3. Provide real-time feedback to users about potential phishing threats

1.8 Project Requirements

In this section, we outline the key requirements and specifications for our phishing detection system, including the actors involved and both functional and non-functional requirements.

1.8.1 Definition of Actors

a) Users

End Users: Individuals who interact with the system to verify URL legitimacy.

User Types: Individuals, company employees, security administrators

Expertise Levels: Beginners (without technical knowledge), intermediates, cybersecurity experts

b) System

Data Management Module: Responsible for collecting, storing, and maintaining the dataset

Learning Module: In charge of training and optimizing models

Monitoring Module: Supervises system performance and detects anomalies

User Interface: Provides an intuitive user experience for URL analysis

1.8.2 Functional Requirements

a) Data Collection

The system must implement web crawlers to extract legitimate URLs from trusted sources. It should utilize phishing data feeds like PhishTank and OpenPhish. API connections with security databases need to be established for enhanced detection. The dataset should be updated daily to identify new phishing techniques. Collection should ensure geographic and linguistic diversity while complying with legal and ethical constraints.

b) Data Preprocessing

A cleaning pipeline must eliminate duplicates using similarity algorithms. It should normalize URLs through lowercase conversion and redirection resolution. The system must handle special characters and international encoding. Anomaly detection techniques should identify outliers. The dataset should be balanced to prevent learning biases and properly partitioned for training, validation, and testing.

c) Feature Extraction

Multiple features must be extracted from each URL. Lexical features include URL length, subdomain count, and special character density. Domain-based features involve WHOIS data, registrar information, and SSL certificate analysis. Page content features examine input forms, JavaScript behavior, and redirection patterns. Reputation features from external services enhance detection accuracy.

d) Model Training

An automated training pipeline should implement hyperparameter optimization techniques. Cross-validation ensures model robustness. Early stopping prevents overfitting. The system should explore ensemble models like Random Forest and LightGBM, neural networks, and hybrid approaches. Multiple evaluation metrics must measure precision, recall, F1 score, and generate detailed error analysis.

e) Model Evaluation

Evaluate models using cross-validation techniques to ensure generalizability. Compare the performance of different models and select the best-performing one.

f) Integration

Integrate the trained model into the phishing detection system. Ensure seamless communication between the model and the front-end interface.

1.8.3 Non-functional requirements

a) Scalability

Design the system to handle a large volume of URL requests efficiently.
Ensure scalability to accommodate future growth in the number of users and URLs.

b) User-Friendly Interface

Design a user-friendly interface for easy interaction with the system. Provide clear and informative feedback to users regarding the phishing risk associated with a given URL. Ensure intuitive navigation and minimize the learning curve for using the system.

c) Performance

The system should be able to process URL scans quickly, even under heavy load. Aim for a response time of milliseconds to ensure a seamless user experience.

1.9 Use Case Diagram

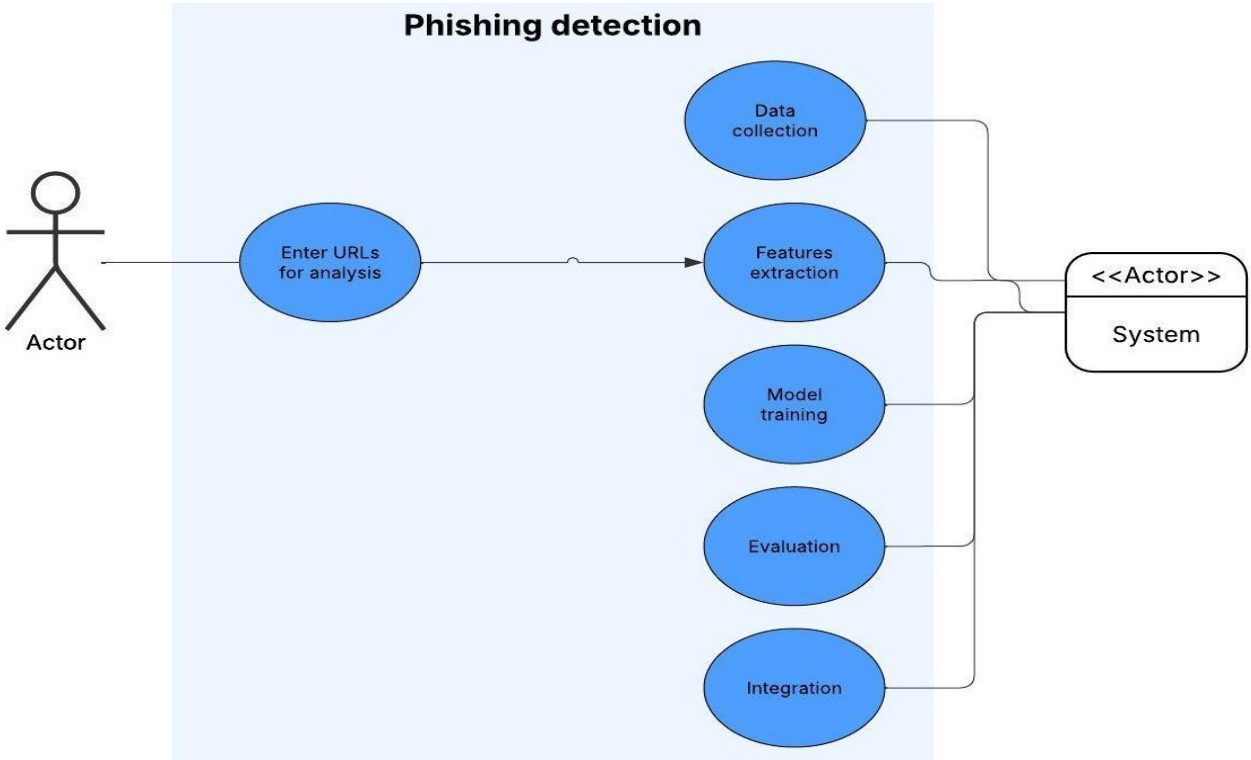


Figure 1.1: UseCase Diagram

The use case diagram depicts the phishing detection system's functionality and actor interactions. A user initiates the process by entering URLs for analysis, while the system handles five key operations: data collection, features extraction, model training, evaluation, and integration.

1.10 Activity Diagram

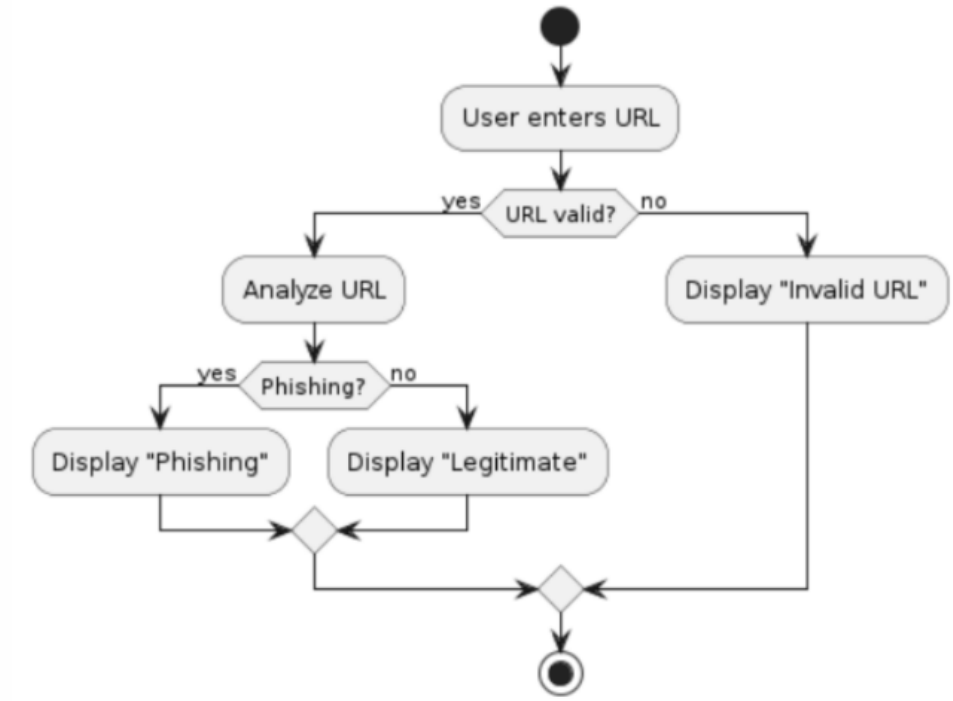


Figure 1.2: Activity Diagram

An activity diagram depicts the process of first verifying if a URL is valid, then determining if it's legitimate or phishing. The diagram showcases two main paths: one for validating URL syntax and another for assessing its legitimacy through phishing detection techniques. This diagram provides a visual representation of the sequential steps involved in the verification process.

1.11 Methodology

To create a data science project, it is necessary to organize the work efficiently, following a structured and iterative approach to achi eve the objectives of the project. In this section, we define the working approach we have adopted and present it in detail.

1.11.1 Crisp-DM Methodology

CRISP-DM (Cross-Industry Standard Process for Data Mining) is a widely adopted methodology that provides a structured approach to guide the entire data mining process.
It offers a comprehensive framework for organizations and data scientists to plan, execute, and manage data mining projects effectively. CRISP-DM emphasizes the iterative and cyclical nature of data mining, allowing for continuous improvement and refinement throughout the project lifecycle. It incorporates key principles such as understanding the business context, collaboration, flexibility, data quality, evaluation, and practical deployment to ensure successful outcomes.

1.11.2 Crisp-DM Stages

The CRISP-DM methodology consists of six major phases:

1. **Business Understanding**: Defining project objectives and requirements from a business perspective

2. **Data Understanding**: Initial data collection and analysis to identify data quality issues and insights

3. **Data Preparation**: Selecting, cleaning, and transforming data for modeling

4. **Modeling**: Selecting and applying various modeling techniques

5. **Evaluation**: Assessing models against business objectives and determining next steps

6. **Deployment**: Implementing the model in the production environment

This cyclical process allows for continuous refinement as new insights emerge during the project lifecycle.

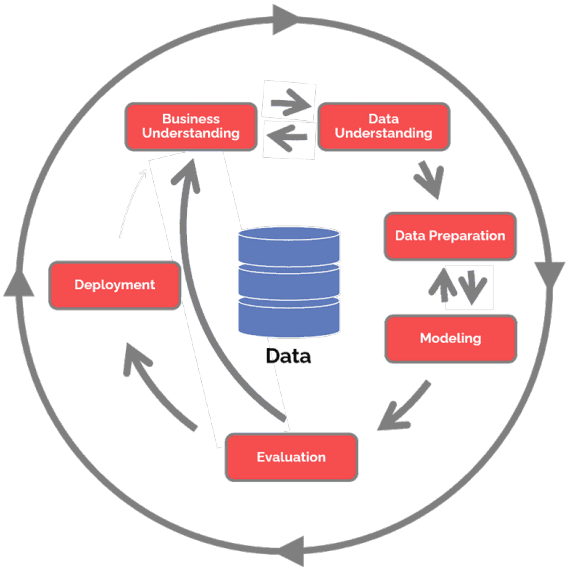


Figure 1.3: CRISP-DM stages [4]

1.12 Conclusion

This chapter provides the essential groundwork for positioning our project within its wider field. We've examined the core issues of phishing attacks, defined the boundaries of our research, and proposed a potential solution pathway leveraging machine learning and deep learning techniques. By outlining the functional and non-functional requirements and defining the system's key actors, we've established a clear foundation for the development of our phishing detection system. The requirements and system design presented establish a structured approach that facilitates a modular, scalable, and robust implementation aligned with the system's objectives. By combining a thorough understanding of the problem domain with detailed specifications, we ensure that our approach is both theoretically sound and practically implementable.

Chapter

2 Methodology

Contents

2.1 Introduction 16

2.2 Project Architecture 16

2.3 Dataset Understanding 17

2.3.1 Dataset Exploration 17

2.3.2 Dataset Selection 18

2.4 Data Preprocessing 20

2.4.1 Data Cleaning 20

2.4.2 Feature Scaling 20

2.4.3 Anomaly Detection 21

2.4.4 Data Balancing 21

2.4.5 Feature Selection 23

2.5 Model Selection and Traning 23

2.5.1 Convolutional Neural Networks Model (CNNs) 23

2.5.2 Random Forest Model 24

2.5.3 Light Gradient Boosting Machine Model (LightGBM) 24

2.5.4 Combination of two Models (LightGBM , CNN) 25

2.5.5 Other Models 25

2.6 Evaluation Metrics 27

2.6.1 Accuracy Metric 27

2.6.2 Precision Metric 27

2.6.3 Recall Metric (Sensitivity) 28

2.6.4 F1-Score Metric 28

2.7 Conclusion 28

2.1 Introduction

In this chapter, we outline our project's architecture and present a focused analysis of the PHISHFOE [33] dataset, created by Yanche Ari Kustiawan and Khairil Imran Ghauth. We highlight its unique features, which are essential for effective phishing detection. Following this, we describe the data preprocessing techniques applied to ensure the dataset's integrity and relevance, laying a strong foundation for further analysis and insights.

2.2 Project Architecture

A well-defined architecture serves as a blueprint that outlines a project's structure, components, and their interactions, ensuring clarity and alignment among all contributors. Figure 3.1 illustrates the flow of a data preprocessing system for machine learning, which involves collecting and understanding data, preprocessing it, selecting and training models, and concluding with evaluation and deployment.

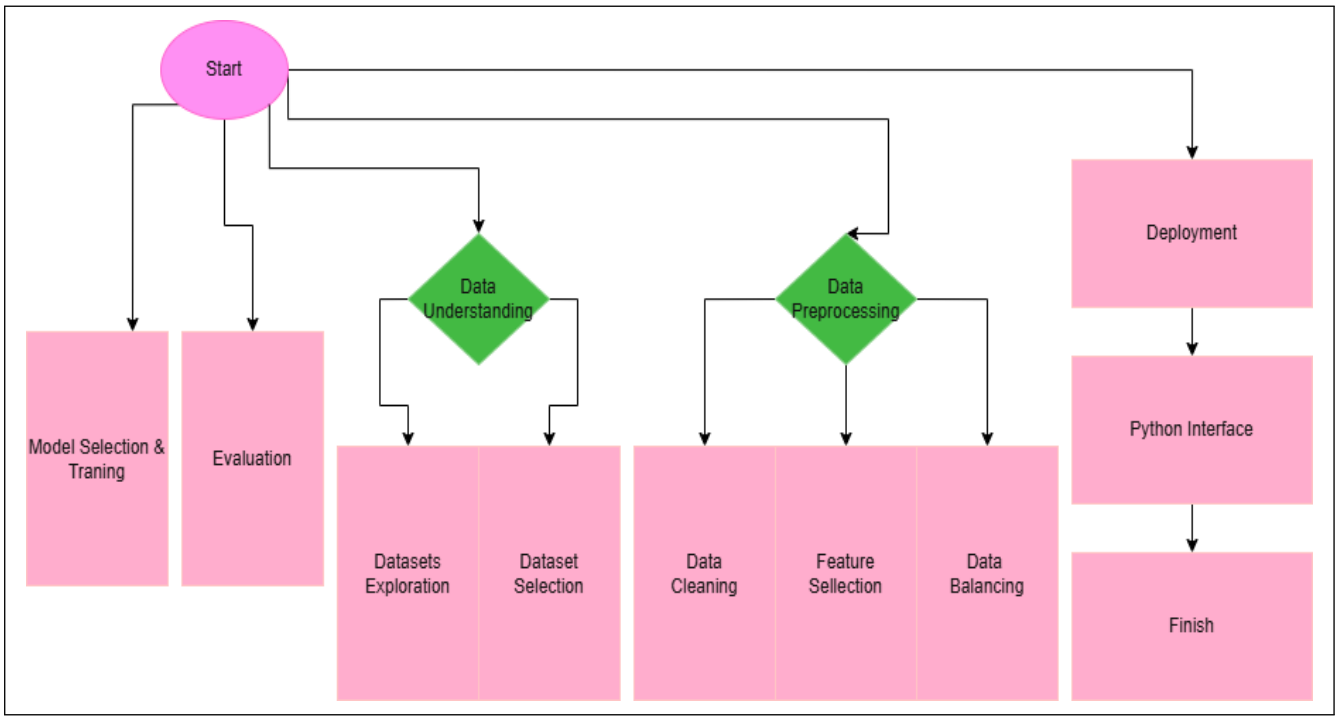


Figure 2.1: Project Architecture Diagram

This figure 2.1 is a flowchart outlining our machine learning pipeline, starting from data understanding and preprocessing to model training, evaluation, and final deployment.

2.3 Dataset Understanding

2.3.1 Dataset Exploration

2.3.1.1 The Web Page Phishing Detection dataset

The Web Page Phishing Detection dataset [17], widely cited in related studies, was compiled in May 2020 from open-source lists such as Alexa and Yandex for legitimate URLs, and OpenPhish and PhishTank for phishing ones. It consists of 11,430 URLs with 87 features and a 'status' label indicating whether a URL is 'legitimate' or 'phishing'. The features are divided into three groups: 56 based on URL structure, 24 from page content, and 7 from external sources. The dataset is clean, with no missing values and only 174 duplicates, making it a strong benchmark for machine learning-based phishing detection systems.

2.3.1.2 Phishing Dataset for Machine Learning: Feature Evaluation

The phishing detection dataset [31] used in our study was compiled by Tan, Choon Lin and published on Mendeley Data in March 2018. It includes 10,000 samples, equally split between phishing and legitimate webpages, collected from PhishTank, OpenPhish, Alexa, and Common Crawl during 2015 and 2017. A key strength of this dataset is the use of Selenium WebDriver for feature extraction, providing more accurate results than traditional parsing methods. It contains 48 numeric features related to URL structure, domain details, and webpage behavior, making it suitable for use in machine learning tools like WEKA. Its balanced structure and feature richness make it ideal for benchmarking and feature evaluation in phishing detection.

2.3.1.3 Phishing Websites UC irvine

The UCI phishing dataset [24] is one of the earliest and most well-known datasets used in phishing detection research. Developed in 2012 by R. Mohammad, F. Thabtah, and L. McCluskey at the University of California, Irvine, it was publicly released in 2015 via the UCI Machine Learning Repository. It contains 30 features such as URL length and page traffic, with values of -1, 0, or 1 indicating website legitimacy levels. The 'Result' column provides the final classification, excluding the value 0. While the dataset has no missing values, it includes 5,205 duplicate entries, which may bias machine learning models and lead to misleading performance outcomes.

2.3.1.4 PhishFOE Dataset

The PhishFOE dataset [33] is a recent and extensive resource developed for phishing detection using machine learning. It comprises 101,083 labeled URLs, with 32 features extracted from both the structure of URLs and the HTML content of webpages. These features include security indicators like HTTPS usage, structural attributes such as subdomain count and obfuscated characters, and HTML-based elements like the presence of JavaScript, popups, or phishing-related keywords (e.g., "bank", "pay", "crypto"). The target label distinguishes phishing (1) from legitimate (0) websites. With no indication of missing values or duplicates, the dataset is well-suited for robust machine learning experiments in phishing website detection and security analysis

2.3.2 Dataset Selection

The decision to integrate the PhishFOE dataset into our project is driven by several strategic considerations, especially when measured against the limitations of alternative datasets. While some resources, such as the Web page phishing detection dataset, provide a robust foundation with a larger number of extracted features and moderate URL counts, others like the Phishing Dataset for Machine Learning: Feature Evaluation offer fewer features, potentially constraining

the depth of analysis required for an effective classifier. In addition, while datasets such as the Phishing Websites UC Irvine may offer a high volume of instances, their prevalence of duplicate entries and a reduced feature set can introduce bias and limit the richness of data for model training. In contrast, as illustrated in the figure 2.2, the PhishFOE dataset comprises an impressive 101,083 URLs paired with 32 carefully extracted features derived from both URL structures and webpage content. This balanced combination not only ensures comprehensive coverage of phishing indicators but also minimizes redundancy, thereby enhancing the overall robustness and accuracy of our phishing detection system.

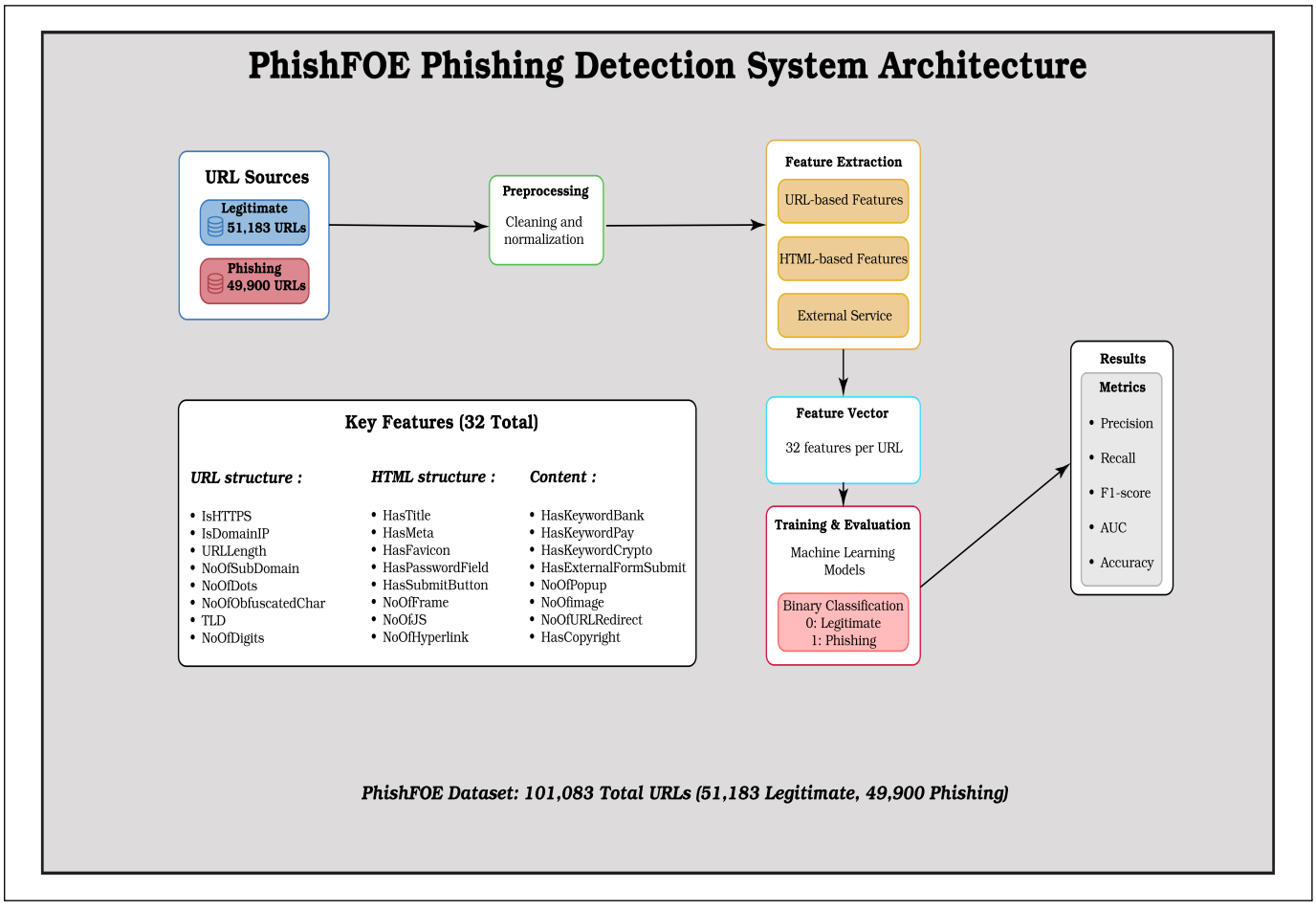


Figure 2.2: Structure of PhishFOE Dataset

Displayed in the figure 2.2 is the PhishFOE system architecture, which collects and processes URLs, extracts features, and uses a machine learning model to classify them as phishing or legitimate.

2.4 Data Preprocessing

Before the data can be used directly by any training algorithm, it must first undergo preprocessing.

2.4.1 Data Cleaning

Inaccuracies and inconsistencies in model performance can stem from the presence of empty, duplicate, or erroneous values within the dataset. Consequently, a data cleaning process was conducted to eliminate such anomalies. The distribution obtained after this preprocessing step is presented in Figure 2.4.

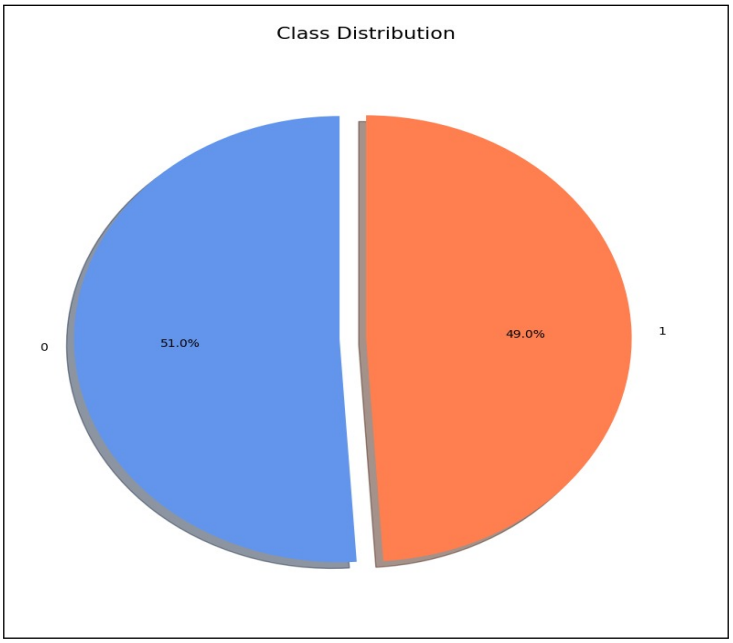


Figure 2.3: Class Distribution

2.4.2 Feature Scaling

Normalization is a preprocessing step that adjusts numerical features to a common scale, ensuring that no single feature disproportionately influences the model due to its magnitude. In our project, we applied Min-Max scaling, which transforms each feature to a specified range typically

between 0 and 1 using the formula:

$$x_{\text{scaled}} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

This technique preserves the original distribution of the data and is particularly effective for algorithms sensitive to feature scales, such as neural networks and k-nearest neighbors.

2.4.3 Anomaly Detection

We applied Isolation Forest [22] (IF) for anomaly detection in phishing detection. Isolation Forest identifies anomalies by isolating observations through random partitioning in an ensemble of binary trees. Points that require fewer splits to isolate are considered more likely to be anomalous. This method enabled the detection of suspicious phishing patterns by assigning anomaly scores to each instance, flagging those with high isolation potential. As an unsupervised approach, it effectively identified outliers without the need for labeled data, improving the robustness of the detection system while maintaining computational efficiency and strong performance on unseen examples.

2.4.4 Data Balancing

To address the issue of class imbalance in our dataset, we employed the Synthetic Minority Over-sampling Technique (SMOTE [10]). This method generates synthetic samples for the minority class by interpolating between existing examples, thereby improving the model's ability to learn from underrepresented data without discarding any majority class instances. By applying SMOTE, we aimed to enhance the classifier's performance and mitigate bias toward the majority class.

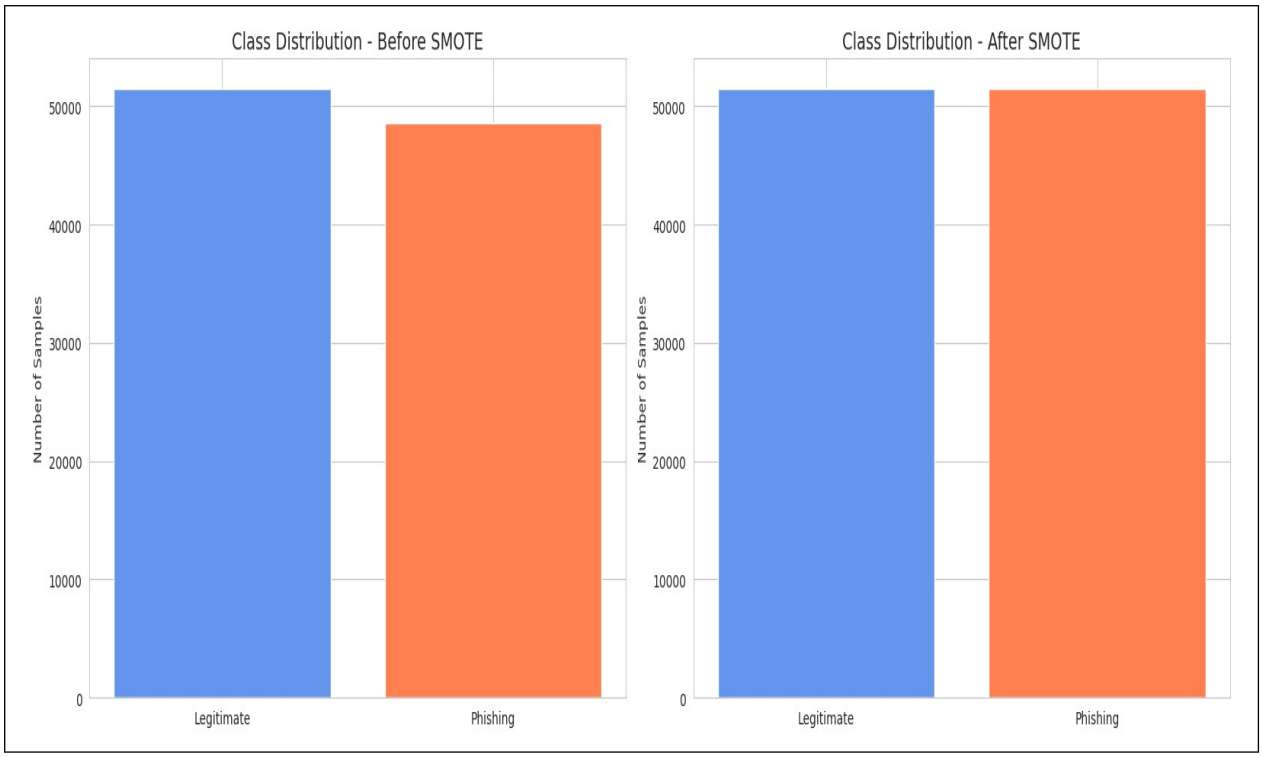


Figure 2.4: Class Distribution Before and After SMOTE

The figure 2.4 shows class distribution before and after applying SMOTE, where the phishing and legitimate classes are more balanced after SMOTE is used to address class imbalance.

2.4.5 Feature Selection

We applied a feature selection strategy based on Recursive Feature Elimination with Cross-Validation (RFE-CV) to identify the most discriminative subset of features from our original pool of 32 indicators. First, all features were power-transformed (PowerTransformer [26]) and standardized (StandardScaler [14]) to ensure comparability. Then, we used a classifier within the RFE-CV [29] framework to iteratively eliminate the least important features while evaluating model performance through cross-validation. This process distilled the feature set down to 23 highly informative predictors: structural URL properties (e.g., IsHTTPS, URLLength, NoOfSubDomain, NoOfDots, NoOfObfuscatedChar), content-based signals (e.g., HasTitle, HasMeta, HasFavicon, HasPasswordField, HasSubmitButton, HasKeywordCrypto), and dynamic page characteristics (e.g., NoOfJS, NoOfCSS, NoOfPopup, NoOfiFrame, NoOfImage, NoOfURLRedirect, NoOfHyperlink). This targeted reduction not only streamlined the model improving interpretability and reducing overfitting risk but also preserved stellar performance, as evidenced by an ROC AUC of 0.999 and a PR AUC of 0.999 on held-out data.

2.5 Model Selection and Traning

Model selection is the process of choosing the most suitable algorithm or architecture based on the problem and data characteristics to ensure effective learning and accurate predictions. Model training involves fitting the selected model to the training data by adjusting its parameters to minimize errors or optimize a specific objective.

2.5.1 Convolutional Neural Networks Model (CNNs)

Convolutional Neural Networks (CNNs) [15] are specialized deep learning architectures that automatically extract hierarchical features from grid-like data such as images by applying trainable convolutional filters across spatial dimensions, followed by pooling and non-linear activation layers. This layered structure enables CNNs to capture increasingly abstract patterns—from simple edges in initial layers to complex object parts in deeper ones—while sharing parameters

and leveraging local connectivity to reduce model complexity and improve generalization. Their remarkable effectiveness in tasks like image classification, object detection, and segmentation arises from this hierarchical feature learning and built-in translation invariance, making CNNs the foundation of state-of-the-art computer vision systems.

2.5.2 Random Forest Model

RandomForest [20] is an ensemble learning algorithmthat employsbagging (bootstrap aggregating) to construct multiple decision trees during training and outputs the class that is the mode of the individual trees' predictions, effectively reducing variance and overfitting. By randomly selecting both observations with replacement and a subset of features for each tree, Random Forest successfully decorrelates the base learners, enhancing model robustness and generalization capability. Through mechanisms like out-of-bag error estimation, embedded feature importance metrics, and parallel tree construction, Random Forest achieves excellent performance on high-dimensional datasets while maintaining interpretability. Its inherent ability to handle non-linear relationships, resistance to outliers, and automatic handling of missing values without requiring extensive preprocessing makes it particularly effective for classification tasks like phishing detection, where it consistently delivers strong performance across diverse real-world applications and benchmarks.

2.5.3 Light Gradient Boosting Machine Model (LightGBM)

LightGBM [21] is a high-performance, open-source gradient boosting framework developed by Microsoft that leverages a histogram-based algorithm, leaf-wise tree growth, and novel techniques like Gradient-Based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB) to dramatically reduce both training time and memory footprint. By growing trees leaf wise always splitting the leaf that yields the largest loss reduction—LightGBM captures intricate patterns more effectively than traditional level-wise methods, leading to faster convergence andbetter generalization. Its built-in support for parallel and GPU acceleration, efficient handling of sparse and categorical features without extensive preprocessing, and robust regularization

make it exceptionally well-suited for large-scale, high-dimensional tasks, often outpacing other boosting algorithms in both accuracy and speed.

2.5.4 Combination of two Models (LightGBM , CNN)

Combining LightGBM and CNN [13] leverages the strengths of both models—CNNs excel at automatically extracting rich, hierarchical features from unstructured data like images, while LightGBM is highly efficient at handling structured, tabular data. In such a hybrid approach, CNNs can be used to extract deep features from images, which are then fed into LightGBM as input for final classification or regression. This fusion benefits from the CNN's ability to learn complex visual representations and LightGBM's strength in fast, accurate decision making on structured data. As a result, the combination often achieves superior performance, especially in multimodal tasks where both image and numerical data are present.

2.5.5 Other Models

2.5.5.1 Transformers

Transformers [32] are advanced deep learning models that utilize self-attention mechanisms to capture long-range dependencies in sequential or structured data. Unlike CNNs, which rely on local receptive fields, Transformers process entire input sequences simultaneously, dynamically weighting the importance of different features through attention scores. In phishing detection, Transformers can analyze URL strings or webpage content by modeling character- or token-level relationships, potentially uncovering subtle malicious patterns missed by traditional methods.

2.5.5.2 Multilayer Perceptron (MLP)

MLPs [16] are classic feedforward neural networks consisting of fully connected layers with non-linear activation functions (e.g., ReLU). While less sophisticated than CNNs or Transformers, MLPs can approximate complex non-linear relationships in tabular data and serve as strong

baselines. For phishing detection, MLPs process handcrafted URL features (e.g., length, special characters) but may struggle with high-dimensional data due to overfitting. Regularization techniques (dropout, weight decay) and architectures like "wide-and-deep" networks can improve their performance.

2.5.5.3 Long Short-Term Memory (LSTM)

LSTMs [18] are a type of recurrent neural network (RNN) designed to capture long-range dependencies in sequential data through gated memory cells. Unlike traditional RNNs, LSTMs mitigate vanishing gradient issues, making them effective for learning temporal patterns. In phishing detection, LSTMs are particularly useful for analyzing sequences such as URLs, email content, or user behavior logs. They can model character-level or token-level sequences, identifying subtle patterns that rule-based models may miss. However, LSTMs require careful tuning and are computationally intensive. Techniques like attention mechanisms, bidirectional layers, and sequence embedding can enhance their representational power and robustness.

2.5.5.4 Extreme Gradient Boosting (XGBoost)

XGBoost [11] is an optimized gradient-boosting framework known for its speed, scalability, and regularization (L1/L2 penalties). It builds trees level-wise (unlike LightGBM's leaf-wise approach) and includes advanced features like sparsity-aware splits and hardware acceleration. XGBoost often outperforms Random Forest on structured data and is widely used in phishing detection due to its interpretability (SHAP values) and robustness to noisy features.

2.5.5.5 Hybrid Model: XGBoost + LSTM

Integrating XGBoost with LSTM leverages XGBoost's efficiency in feature selection and LSTM's capability to model sequential or contextual dependencies [18]. In this ensemble:

- **XGBoost** selects and transforms important features from the dataset.

- **LSTM** receives the refined feature vectors (or probability outputs) as sequences and learns temporal or pattern-based representations for final prediction.

This hybrid configuration aims to capture both structural and temporal patterns in phishing data. XGBoost reduces feature redundancy and enhances input quality, while LSTM brings depth to pattern recognition—especially for features derived from URL sequences or email text. Strategies like cascading or attention-weighted fusion can improve overall robustness and interpretability.

2.6 Evaluation Metrics

Evaluation metrics [12] are essential for measuring the effectiveness of machine learning models, particularly in classification tasks. They provide a deeper understanding of a model's strengths and limitations. Below are the key metrics used in our evaluation:

2.6.1 Accuracy Metric

Accuracy measures the proportion of correctly classified instances out of the total predictions made. It is calculated as:

$$Accuracy = \frac{Number\ of\ Correct\ Predictions}{Total\ Number\ of\ Predictions}$$

While accuracy offers a general overview of model performance, it can be misleading when dealing with imbalanced datasets.

2.6.2 Precision Metric

Precision indicates the proportion of true positive predictions among all instances predicted as positive. It is defined as:

$$Presicion = \frac{True\ Positives}{True\ Positives\ +\ Flase\ Positives}$$

Precision is particularly important in scenarios where false positives are costly.

2.6.3 Recall Metric (Sensitivity)

Recall, also known as sensitivity or the true positive rate, measures the ability of the model to correctly identify all actual positive instances. It is computed as:

$$Recall = \frac{True\ Positives}{True\ Positives\ +\ Flase\ Negatives}$$

Recall is crucial when the cost of missing positive cases (false negatives) is high.

2.6.4 F1-Score Metric

The F1-score is the harmonic mean of precision and recall, providing a single measure that balances both metrics. It is especially useful when working with imbalanced datasets. The formula is:

$$F1 - Score = 2\ \times\ \frac{Precision\ \times\ Recall}{Precision\ +\ Recall}$$

The F1-score ranges from 0 to 1, where a higher value indicates better overall model performance in terms of both precision and recall.

2.7 Conclusion

In this chapter, we presented the overall architecture of the project and described the datasets utilized. We detailed the data preprocessing steps and outlined the various machine learning models explored in our effort to build an effective and intelligent Phishing detection system. In the following chapter, we will discuss the results obtained from training these models and the deployment strategies implemented for our intrusion detection system.

Chapter

3 Experimental Results and Deployment

Contents

3.1 INTRODUCTION 30

3.2 Software Environment 30

3.2.1 Development environment : Google Colab 30

3.2.2 Development environment : Visual Studio Code 30

3.2.3 Python Library : Streamlit 30

3.3 Results and Evaluation 31

3.3.1 Balancing Techniques 31

3.3.2 Models Details 32

3.4 Feature Selection Methods 33

3.4.1 Combining Models with RFE-CV 33

3.4.2 Combining Models with f-classif 34

3.4.3 Comparing f-classif and RFE-CV 34

3.5 Other Results 35

3.5.1 Comparing our results with other studies 35

3.5.2 Additional Models Tested on The phishFOE dataset 36

3.6 Deployment 37

3.6.1 System Workflow Overview 37

3.6.2 Deployment Environment Preparation 38

3.6.3 User Interface and Prediction Output 39

3.6.4 Notable Technical Features 41

3.6.5 Conclusion 42

3.1 INTRODUCTION

This chapter focuses on presenting the evaluation results of various algorithms discussed in the previous chapter. Through this analysis and comparison, we aim to identify the most efficient model for our database. Subsequently, we will proceed with deploying our solution.

3.2 Software Environment

3.2.1 Development environment : Google Colab

Google Colab [5], a cloud-hosted development environment offered by Google, enables the execution and creation of Python code via a web interface. Users can generate and run Jupyter notebook files, integrating code, text, and visualizations. It includes complimentary GPU access and 12.7 GB of RAM. In our project focusing on machine learning-based phishing de- tection, Google Colab serves as the central platform for preprocessing, model assessment, and feature selection tasks.

3.2.2 Development environment : Visual Studio Code

Visual Studio Code [6] (VS Code) is a lightweight, open-source code editor developed by Microsoft. It supports multiple programming languages, offers powerful extensions, integrated Git, debugging tools, and a user-friendly interface, making it ideal for efficient development workflows.

3.2.3 Python Library : Streamlit

Streamlit [30] is an open-source Python library used for building interactive web applications and data dashboards. It simplifies the process of creating and sharing data-driven applications by providing an easy-to-use interface and a fast development workflow.

3.3 Results and Evaluation

3.3.1 Balancing Techniques

Following the resolution of data duplication, our subsequent focus was on addressing the class imbalance within our dataset. We observed a discrepancy of 2081 more instances of legitimate cases than phishing ones. To address this, we investigated two common balancing methods, SMOTE (Synthetic Minority Over-sampling Technique) and undersampling, before deciding not to use SMOTE method. We employed four different models for our analysis: Random Forest,LightGBM, CNN and a combination (LightGBM , CNN). Initially, we applied these models without any balancing technique and observed the results. Regardless of the balancing approach used, the SMOTE method gave the best performance metrics.

Table 3.1: Performance Metrics for Different Algorithms and Sampling Techniques

Sampling	Algorithm	Metrics
None	Random Forest	*Acc: 0.9750, Prec: 0.9815, Rec: 0.9750, F1: 0.9750*
	LightGBM	***Acc: 0.9885, Prec: 0.9902, Rec: 0.9885, F1: 0.9885***
	CNN	*Acc: 0.9650, Prec: 0.9652, Rec: 0.9650, F1: 0.9650*
	LightGBM+CNN	*Acc: 0.9792, Prec: 0.9850, Rec: 0.9792, F1: 0.9792*
SMOTE	Random Forest	*Acc: 0.9805, Prec: 0.9916, Rec: 0.9899, F1: 0.9899*
	LightGBM	***Acc: 0.9905, Prec: 0.9931, Rec: 0.9879, F1: 0.9905***
	CNN	*Acc: 0.9754, Prec: 0.9755, Rec: 0.9754, F1: 0.9754*
	LightGBM+CNN	*Acc: 0.9850, Prec: 0.9850, Rec: 0.9850, F1: 0.9850*
Undersampling	Random Forest	*Acc: 0.9789, Prec: 0.9805, Rec: 0.9789, F1: 0.9789*
	LightGBM	***Acc: 0.9886, Prec: 0.9905, Rec: 0.9916, F1: 0.9886***
	CNN	*Acc: 0.9675, Prec: 0.9671, Rec: 0.9675, F1: 0.9675*
	LightGBM+CNN	*Acc: 0.9850, Prec: 0.9851, Rec: 0.9850, F1: 0.9850*

This table 3.1 compares classification performance across models and sampling techniques.

3.3.2 Models Details

Table 3.2: Parameters used for each model

Model	Parameters
Random Forest	- ‘max_depth’: 10 - ‘min_samples_split’: 5 - ‘min_samples_leaf’: 4 - ‘random_state’: 42 - ‘n_estimators’: 100 - ‘verbose’: 1
LightGBM	- ‘num_leaves’: 31 - ‘learning_rate’: 0.05 - ‘n_estimators’: 100 - ‘boosting_type’: ‘gbdt’ - ‘objective’: ‘binary’ - ‘max_depth’: -1
CNN	- ‘epochs’: 50 - ‘batch_size’: 64 - ‘optimizer’: ‘adam’ - ‘loss’: ‘binary_crossentropy’ - ‘activation’: ‘relu’ (hidden), ‘sigmoid’ (output) - ‘dropout’: 0.2
LightGBM+CNN	- ‘lightgbm_params’: same as LightGBM above - ‘cnn_layers’: 2 convolutional + 2 dense - ‘fusion_strategy’: concatenate outputs - ‘optimizer’: ‘adam’ - ‘epochs’: 64 - ‘batch_size’: 30

3.4 Feature Selection Methods

Feature selection is a pivotal step in machine learning, focusing on identifying the most influential features within a dataset while discarding irrelevant or redundant ones. In this experiment, wrapper-based algorithms are utilized to systematically select features based solely on their predictive power. The selected features undergo evaluation using various methods,including f-classif [28] and RFE-CV [29]. By comparing the performance of these techniques, we aim to determine the most effective approach for feature selection.

3.4.1 Combining Models with RFE-CV

We display through 3.1, the results when employing different models in conjunction with RFE-CV.

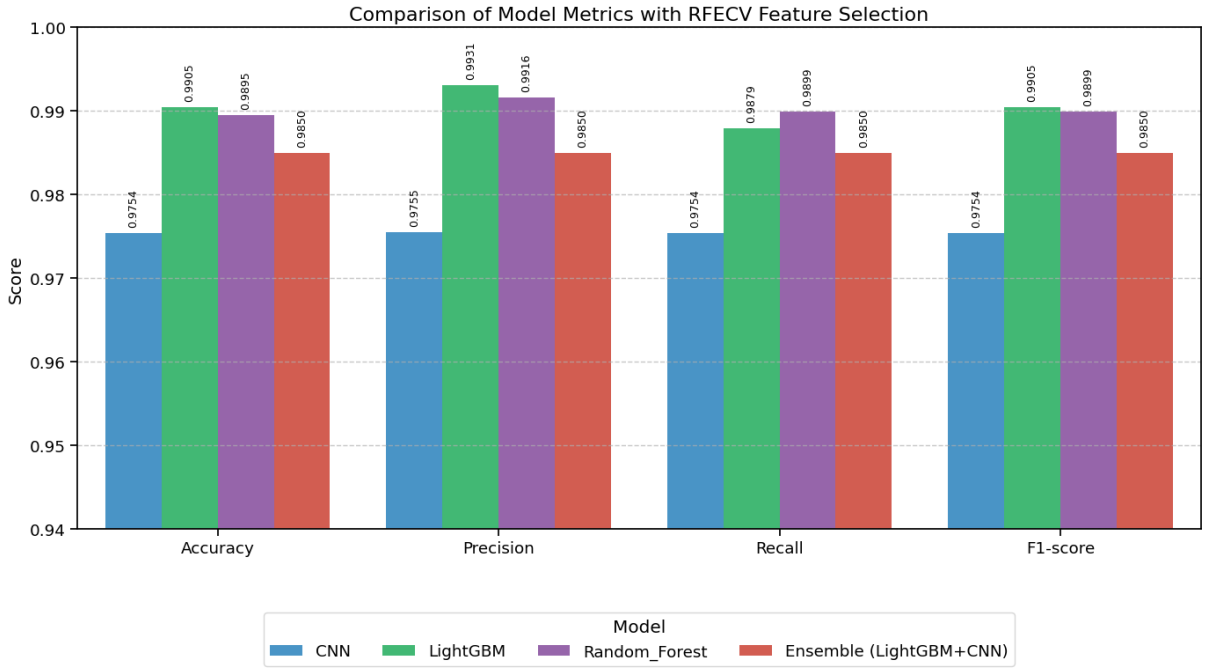


Figure 3.1: Activity Diagram

The image 3.1 displays a comparison of models and shows that LightGBM outperforms other models after RFECV feature selection, achieving the highest accuracy, precision, recall, and F1-score.

3.4.2 Combining Models with f-classif

We display through 3.2, the results when employing different models in conjunction with f-classif.

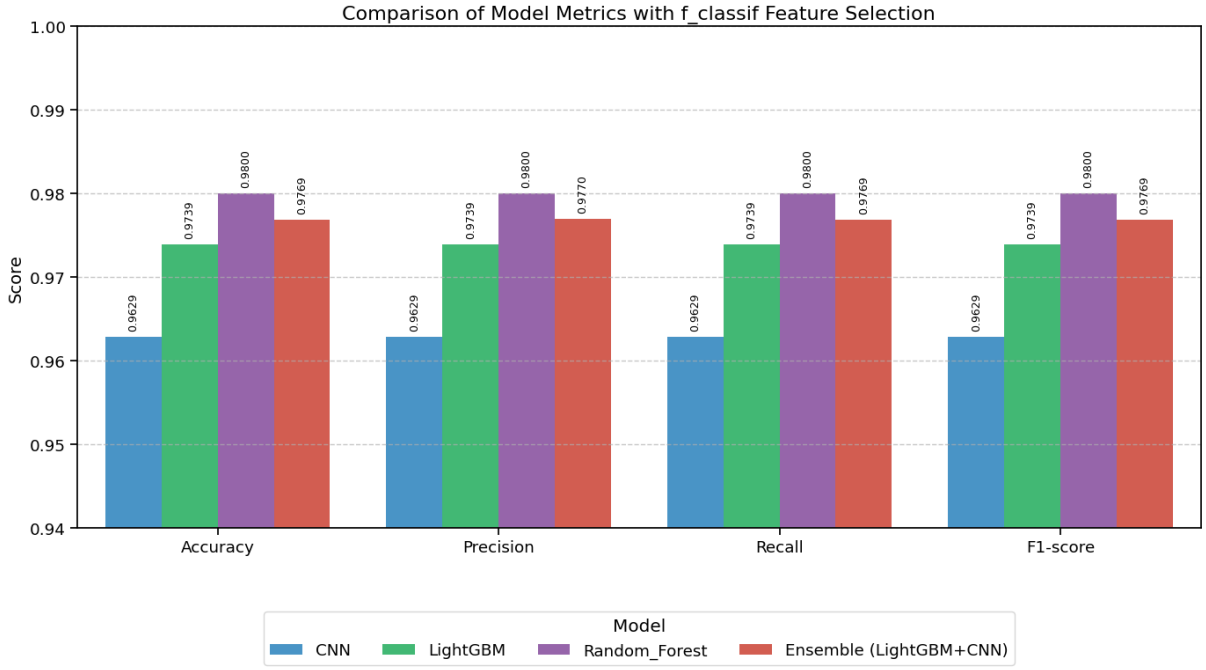


Figure 3.2: Performance comparison of various models using f-classif

The image 3.2 displays a comparison of models and shows that Random Forest outperforms other models after RFECV feature selection, achieving the highest accuracy, precision, recall, and F1-score.

3.4.3 Comparing f-classif and RFE-CV

We display through 3.3, the results when employing different models in comparing f-classif with RFE-CV.

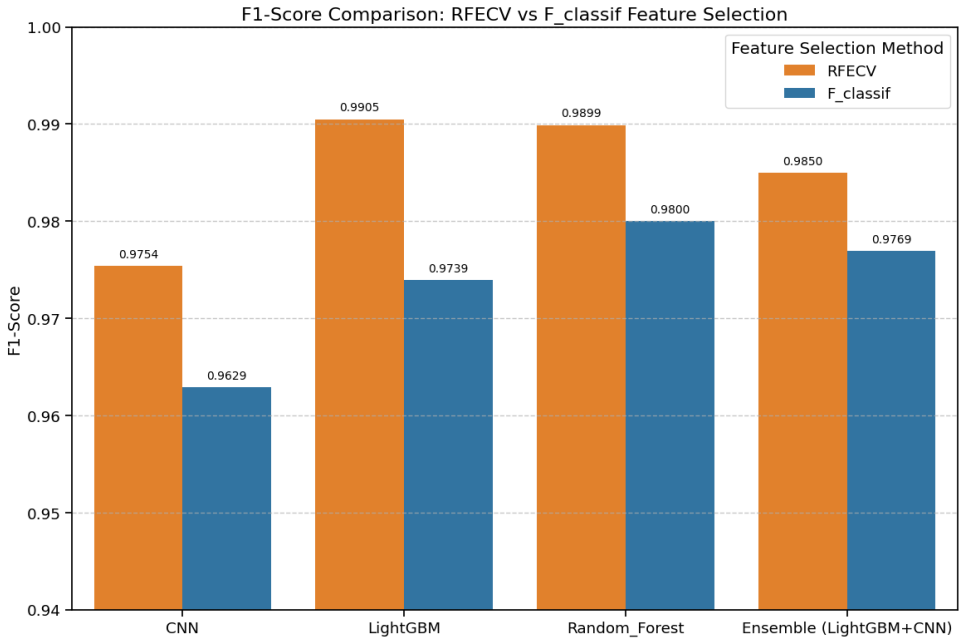


Figure 3.3: Comparaison between f-classif and RFE-CV

The figure 3.3 compares F1-scores of four models using two feature selection methods: RFECV and f classif. Across all models, f classif yields higher F1-scores. Notably, Random Forest and LightGBM perform best with f classif, achieving 0.9899 and 0.9903 respectively, indicating f classif's superior effectiveness in this setup.

3.5 Other Results

3.5.1 Comparing our results with other studies

The comparative analysis demonstrates that modern machine learning models achieve high performance in phishing detection, with accuracy scores consistently exceeding 90% across studies. While existing approaches like XGBoost (97.24%) and RF (96.83%) show strong results on benchmark datasets (UCI, Web Phishing), our proposed LightGBM model with RFE-CV feature selection achieves superior performance (98.90% accuracy, 0.9890 F1) on the phishFOE dataset. This improvement suggests that recursive feature elimination with cross-validation effectively optimizes feature subsets for detection tasks. The results underscore the importance

Table 3.3: Extended Comparison of Machine Learning Models for Phishing Detection Across Different Datasets

Study	Feature Selection	#Feat.	Models	Dataset	Accuracy / F1 Score
		UCI Dataset			
Liyana et al. [27] (2022)	IG, TreeSHAP	15	XGBoost	UCI	91.05% / 91.5%
Moedjahedy et al. [23] (2021)	RFE + Corr.	10	AdaBoost	UCI	94.51% / 94.2%
Al-Mohammed et al. [7] (2024)	Mutual Info	25	LightGBM	UCI	95.12% / 94.7%
Our Work	RFE-CV	23	LightGBM	UCI	**97.07% / 97.07%**
Our Work	RFE-CV	23	XGBoost	UCI	**97.03% / 96.98%**
Our Work	RFE-CV	23	CNN	UCI	**96.50% / 96.50%**
		Phishing_Legitimate_full_codes Dataset			
Ahmad et al. (2024)	Correlation Heatmap	30	RF, SVM	PLFC	97.2% / 97.1%
Khan et al. (2024)	PCA, Chi-Square	25	CNN, RNN, RF	PLFC	97.8% / 97.6%
Our Work	RFE-CV	23	LightGBM	PLFC	**99.02% / 98.89%**
Our Work	RFE-CV	23	Random Forest	PLFC	**98.70% / 98.85%**
		Phishing_Detection_Dataset			
Almsedin et al. [8] (2019)	Not specified	20	RF, MLP, J48	PDD	98.11% / –
Mat Rani et al. [9] (2023)	TreeSHAP, Info Gain	15	XGBoost, RF, NB	PDD	97.8% / –
Our Work	RFE-CV	23	CNN	PDD	**97.50% / 95.50%**
Our Work	RFE-CV	23	LightGBM	PDD	**98.90% / 98.90%**
Our Work	RFE-CV	23	MLP	PDD	**96.80% / 9680%**

of tailored feature selection methods alongside advanced algorithms, particularly for emerging phishing datasets that may exhibit unique patterns.

3.5.2 Additional Models Tested on The phishFOE dataset

To further evaluate the effectiveness of different machine learning approaches, table 3.4 presents the performance metrics obtained from additional models tested during experimentation.

Table 3.4: Performance Metrics for Different Models and Techniques

Model	Techniques	Accuracy	F1-score	Precision	Recall
XGBoost	SMOTE + f_classif	0.9854	0.9853	0.9884	0.9822
XGBoost+LSTM	SMOTE + RFE-CV	0.9891	0.9891	0.9900	0.9882
LSTM	SMOTE + ADAM	0.9769	0.9764	0.9766	0.9775
Roberta Transformers	-	0.9405	0.9381	0.9644	0.9131
MLP	SMOTE + SGD	0.9536	0.9536	0.9482	0.9425

3.6 Deployment

Deploying machine learning models for phishing website detection demands a rigorous and structured approach. Our work adheres to the Cross Industry Standard Process for Data Mining (CRISP-DM), ensuring that deployment is not only technically sound but also reliable, secure, and user-friendly.

The deployment phase transitions our model from a theoretical prototype to a fully operational system capable of delivering real-time predictions and actionable insights. Our system architecture encompasses both backend model logic and a responsive frontend interface.

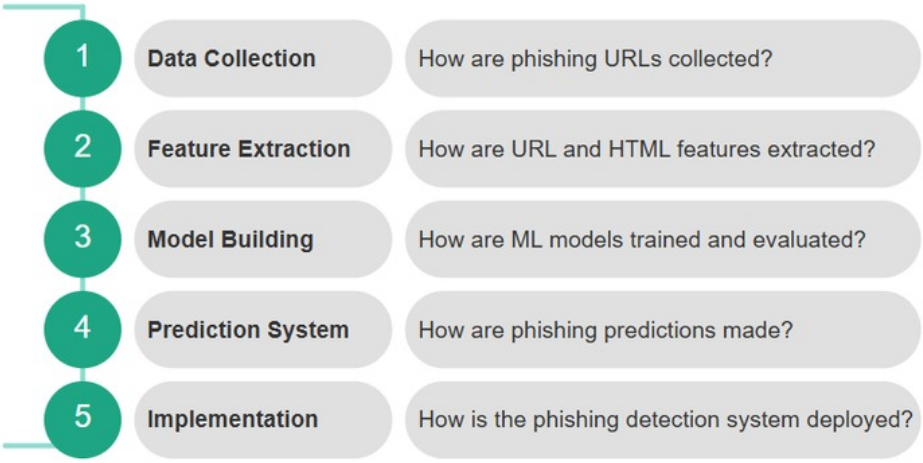


Figure 3.4: Deployment Design

3.6.1 System Workflow Overview

The phishing detection pipeline follows a five-stage architecture:

- **User Input**: Submission of a suspicious URL through the Streamlit-based interface.
- **Feature Extraction**: Extraction of URL and HTML content features:
  - *URL-based features*: Protocol type, URL length, number of subdomains, suspicious characters (e.g., obfuscated symbols), use of shortening services, presence of sensitive keywords, and domain age using WHOIS information.

  - *HTML-based features*: Presence of titles, meta tags, favicon, copyright mentions, social media links, password fields, cryptocurrency-related terms, popups, iframes, and JavaScript redirects.

- **Feature Preprocessing**: Data cleaning and preparation including missing value handling, normalization using PowerTransformer, standardization with StandardScaler, and feature selection to retain only relevant features.

- **Model Prediction (LightGBM)**: Preprocessed features are fed to the trained LightGBM model to output:

  - Raw prediction scores converted to probabilities.
  - Binary classification (0 = legitimate, 1 = phishing).
  - Legitimacy score calculated as $1 - P(\text{phishing})$, displayed as a percentage.

- **Results and Visualization**: Analysis of key features, detection of risky combinations (e.g., password field without HTTPS), display of recommendations, and visualization through an interactive gauge.

3.6.2 Deployment Environment Preparation

Environment Configuration

- **Python Environment**: Isolated via virtual environments.
- **Dependencies**: Managed through a `requirements.txt` file.
- **Configuration Files**: Centralized and environment-specific.

Resource Organization

- **Project Structure**: Logical separation of application code, model files, and static assets.
- **Model Versioning**: Proper handling and storage of trained model artifacts.
- **Static Assets**: Management of CSS, JavaScript, and image files for the frontend interface.

Compatibility and Testing

- **Version Resolution**: Alignment between development and production library versions.
- **Cross-platform Testing**: Ensuring system compatibility across different operating systems.
- **Browser Support**: Verification across major modern web browsers.

Validation checkpoints were used to ensure correct behavior before full deployment.

3.6.3 User Interface and Prediction Output

We developed a user-friendly Streamlit interface allowing users to input URLs and instantly receive legitimacy predictions.

Model Performance Dashboard

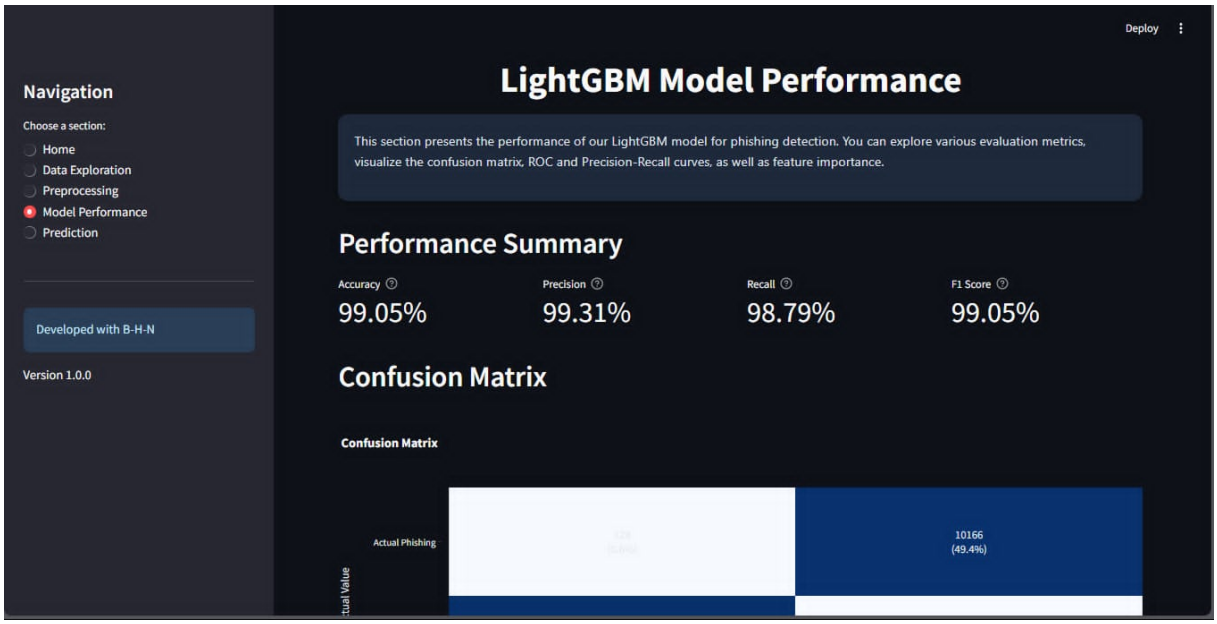


Figure 3.5: LightGBM Model Performance Dashboard

The dashboard displays key evaluation metrics:

- Accuracy: 99.05%

- Precision: 99.31%
- Recall: 98.79%
- F1 Score: 99.05%

Prediction Output Examples

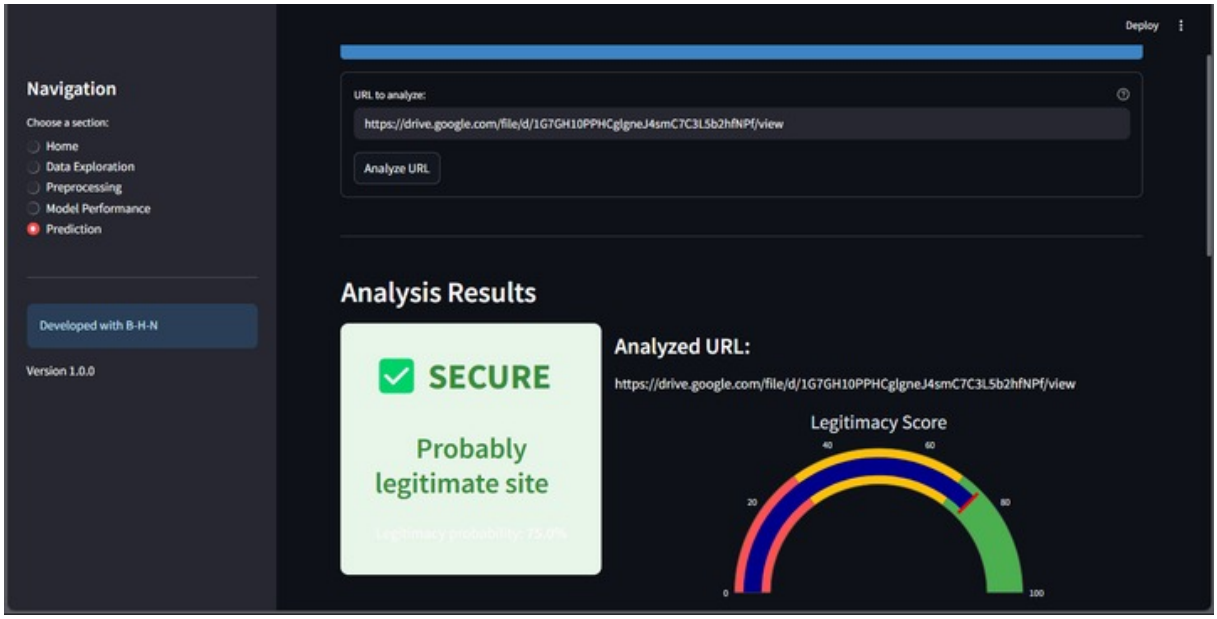


Figure 3.6: URL Legitimacy Analysis Result – Legitimate URL

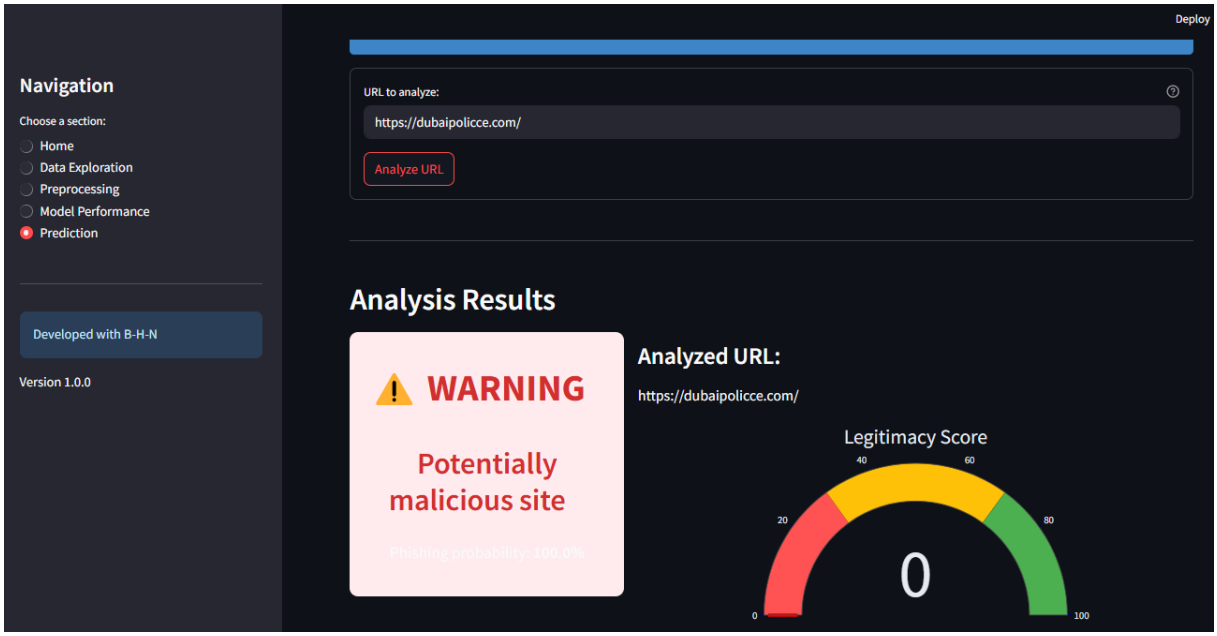


Figure 3.7: URL Legitimacy Analysis Result – Phishing URL

URL Legitimacy Prediction Dashboard



Figure 3.8: URL Legitimacy Prediction Dashboard

The dashboard highlights:

- Extracted feature details.
- Major risk indicators influencing the decision.
- Security tips based on risk analysis.

3.6.4 Notable Technical Features

Several technical enhancements were implemented to ensure performance, reliability, and user experience:

- **Model caching**: Utilization of `@st.cache_resource` to avoid reloading models unnecessarily.
- **Robust error handling**: Detection and management of errors at every pipeline stage.
- **Real-time interface feedback**: Progress bars and status messages to inform users.
- **Detailed logging**: Recording operations for thorough tracking and debugging.
- **Scalable architecture**: Design supporting future upgrades and extensions.

3.6.5 Conclusion

Our phishing detection system deployment follows the CRISP-DM methodology, ensuring a smooth transition from model development to practical use. With a modular, scalable architecture, the system offers intuitive user interaction, efficient predictions, and reliable monitoring.
Well-managed dependencies, organized configurations, and thorough compatibility testing enhance stability and maintainability. Resilient design through advanced feature extraction, robust error handling, and fault tolerance ensures consistent service, even during partial failures.
This approach meets modern technical standards while promoting trust, transparency, and user confidence.

GENERAL CONCLUSION

This project aimed to address the critical and growing issue of phishing attacks by leveraging the power of machine learning to create an intelligent and efficient detection system. Through a structured approach following the CRISP-DM methodology, we explored the full cycle of building a phishing detection system—from understanding the problem domain and gathering data, to implementing and evaluating various models, and finally deploying a functional web application.

We conducted thorough preprocessing on multiple datasets, selected the most relevant features, and applied advanced balancing and anomaly detection techniques to improve model reliability. Among the models tested, LightGBM and its hybrid version with CNN consistently delivered high performance, achieving accuracy, precision, recall, and F1-scores exceeding 98% when balanced with SMOTE.

Deployment of the system via Streamlit enabled real-time URL analysis with a user-friendly interface, making it accessible to non-technical users. The modular architecture, along with robust error-handling strategies, ensured stability and scalability of the application.

Overall, this project demonstrates how machine learning can be effectively applied to cybersecurity problems, providing proactive defense mechanisms against phishing threats. It not only contributes a practical tool for online safety but also enhances understanding of real-world AI applications, offering valuable insights for future developments in secure digital systems.

Bibliography

[1] Black List Detection: https://www.researchgate.net/publication/388567661_Phishing_Detection_Techniques_A_review.

[2] DNS Filtring: https://www.researchgate.net/publication/370209654_Phishing_Attack_Detecting_System_Using_DNS_and_IP_Filtering.

[3] Rule Based: https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2024.1428013/full.

[4] CRISP-DM: https://www.datascience-pm.com/crisp-dm-2/.

[5] URL: https://colab.research.google.com/.

[6] URL: https://code.visualstudio.com/.

[7] Al-Mohammed et al. *High-performance phishing URL classification via mutual information-based feature selection and LightGBM. IEEE Access*. 2024.

[8] Almsedin et al. *Phishing Detection Based on Machine Learning and Feature Selection Methods*. https://online-journals.org/index.php/i-jim/article/view/11411. International Journal of Interactive Mobile Technologies. 2019.

[9] Mat Rani et al. *Feature Selection to Enhance Phishing Website Detection Based On URL Using Machine Learning Techniques*. https://publisher.uthm.edu.my/ojs/index.php/jscdm/article/view/13484. Journal of Soft Computing and Data Mining. 2023.

[10] Bellinger C. Drummond C. and Japkowicz N. *SMOTE :* https://www.sciencedirect.com/science/article/abs/pii/S0020025519306838. A Comprehensive Analysis of Synthetic Minority Oversampling Technique (SMOTE) for High-Dimensional Data. Information Sciences. 2019.

[11] Tianqi Chen and Carlos Guestrin. “Xgboost: A scalable tree boosting system”. In: *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (2016), pp. 785–794.

[12] CSBranch. *Evaluation Metrics*. https://csbranch.com/index.php/2024/11/16/model-evaluation-metrics-accuracy-precision-recall-and-f1-score/.

[13] Nagwa Badr & Mohamed Tolba Esraa AbdElraouf Mohammed Abdel-Megeed. *Combination of Convolutional Neural Network and LightGBM*. https://www.researchgate.net/publication/372945956_An_Efficient_Combination_of_Convolutional_Neural_Network_and_LightGBM_Algorithm_for_Lung_Cancer_Histopathology_Classification. 2023.

[14] Pedregosa F. Varoquaux G. *StandardScaler :* https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html. StandardScaler Overview. 2011.

[15] Geeks For Geeks. *Convolutional Neural Networks Model (CNN)*. https://www.geeksforgeeks.org/convolutional-neural-network-cnn-in-machine-learning/. 2025.

[16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

[17] Abdelhakim Hannousse and Salima Yahiouche. *Web Phishing Dataset :* https://www.kaggle.com/datasets/shashwatwork/web-page-phishing-detection-dataset. Web page phishing detection. 2021.

[18] S. Hochreiter and J. Schmidhuber. *Long Short-Term Memory LSTM*. https://doi.org/10.1162/neco.1997.9.8.1735. Neural Computation.

[19] UC Berkeley School of Information. *Machine Learning*. https://ischoolonline.berkeley.edu/blog/what-is-machine-learning/. 2020.

[20] University of California Berkeley Leo Breiman. *Random Forest*. https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf. 2001.

[21] LightGBM. *Light Gradient Boosting Machine Model LightGBM*. https://lightgbm.readthedocs.io/en/latest/Features.html.

[22] M. Ting Liu .K. “Isolation Forest”. In: *Eighth IEEE International Conference on Data* (2008).

[23] I. Moedjahedy. *Hybrid feature selection for phishing detection using recursive elimination and correlation analysis with AdaBoost. Computers Security, 110,* 2021.

[24] Rami Mohammad and Lee McCluskey. *Phishing Websites UCI :* https://archive.ics.uci.edu/dataset/327/phishing+websites. UCI Machine Learning Repository. 2015.

[25] Rising O. Odegua. *An Empirical Study of Ensemble Techniques (Bagging, Boosting and Stacking)*. https://www.researchgate.net/profile/Rising-Odegua/publication/338681864_An_Empirical_Study_of_Ensemble_Techniques_Bagging_Boosting_and_Stacking/links/5e2417c2458515ba2092f5cf/An-Empirical-Study-of-Ensemble-Techniques-Bagging-Boosting-and-Stacking.pdf. 2019.

[26] *PowerTransformer Documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PowerTransformer.html. PowerTransformer Overview.

[27] Liyana S. *Journal of Cybersecurity, 8(2), 145-160*. 2022.

[28] Scikit-learn. *F-Classif Documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.f_classif.html.

[29] Scikit-learn. *RFE-CV Documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.RFECV.html.

[30] *Streamlit Documentation*. https://docs.streamlit.io.

[31] Choon Lin Tan. *Phishing Full Legitimate Dataset :* https://data.mendeley.com/datasets/h3cgnj8hft/1. Phishing Dataset for Machine Learning: Feature Evaluation. 2018.

[32] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).

[33] Kustiawan YancheAri and ImranGhauth Khairil. *PhishFOE Dataset :* https://ieee-dataport.org/documents/phishfoe-dataset. PhishFOE Dataset Documentation. 2025.

[34] Yoshua Bengio Yann LeCun and Geoffrey Hinton. *Deep learning*. https://www.nature.com/articles/nature14539. 2015.